

Dataset used: [Garbage Images Dataset \(2000/class\)](#).

✓ Mount Google Drive

Mount Google Drive to access and store dataset archives and model weights directly from storage.

```
# Mount Google Drive so we can access the dataset
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

Mounted at /content/drive

✓ Prepare Dataset

Copy the dataset archive from Google Drive to the local Colab environment and extract its contents.

```
from pathlib import Path # handle filesystem paths
# automatically handles path slashes (/ on mac/linux vs \ on windows)

# Define the location of the dataset ZIP file in Google Drive
source_path = Path('/content/drive/MyDrive/AI-ML Folder/ai-ml-zafar-iqbal')

destination_path= Path('/content/dataset/')

# check if file exists at source path
if not source_path.is_file():
    print(f"Error: could not find file at {source_path}")
else:
    print("File found")

# create the directory and any missing parent directories
destination_path.mkdir(parents=True, exist_ok=True)
print(f"Directory created at {destination_path}")
```

File found
Directory created at /content/dataset

```
# copying files from drive
import shutil # shell utilities library for high-level file operations

shutil.copy(source_path, destination_path)
print("File copied from drive successfully!")
```

File copied from drive successfully!

```
import zipfile

zip_file_path = '/content/dataset/archive-.zip'

# Extract the ZIP file so the images can be used for training
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall('/content/archive')

print("Extracted successfully!")
```

Extracted successfully!

✓ Structure Dataset for Ultralytics

Split the raw dataset into `train` (80%) and `val` (20%) directories as required by the Ultralytics YOLO classification format.

```
!pip install split-folders[full]
```

Requirement already satisfied: split-folders[full] in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from split-folders[full])

```
import splitfolders

input_folder = '/content/archive/Garbage_Dataset_Classification/images'
output_folder = '/content/garbage_dataset'

# Split the dataset into training and validation sets
# 80% is used for training and 20% for validation
splitfolders.ratio(
    input_folder,
    output_folder,
    seed=42,
    ratio=(0.8, 0.2),
    group_prefix=None
)
```

Copying files: 13901 files [00:04, 2890.28 files/s]

✓ Train YOLO Classification Model

Install `ultralytics`, initialize the `yolo26s-cls` pre-trained classification model, and train it on the waste dataset.

```
# Install the Ultralytics library for YOLO
!pip install -q ultralytics
```

```
# Import YOLO
from ultralytics import YOLO
```

```
# Load a pre-trained YOLO classification model
model = YOLO("yolo26s-cls")
```

```
# Train the model on the waste classification dataset
model.train(
    data="/content/garbage_dataset",
    epochs=20,
    imgsz=224,
    batch=32,
    workers=2
)
```

```
.4.118 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
r: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugument, batch=32
nt/garbage_dataset/train... found 11120 images in 6 classes ✅
/garbage_dataset/val... found 2781 images in 6 classes ✅
```

```
del.yaml1 nc=1000 with nc=6
```

	from	n	params	module	arguments
	-1	1	928	ultralytics.nn.modules.conv.Conv	[3, 32, 3, 2]
	-1	1	18560	ultralytics.nn.modules.conv.Conv	[32, 64, 3, 2]
	-1	1	26080	ultralytics.nn.modules.block.C3k2	[64, 128, 1, False, 0.01]
	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
	-1	1	103360	ultralytics.nn.modules.block.C3k2	[128, 256, 1, False, 0.01]
	-1	1	590336	ultralytics.nn.modules.conv.Conv	[256, 256, 3, 2]
	-1	1	346112	ultralytics.nn.modules.block.C3k2	[256, 256, 1, True]
	-1	1	1180672	ultralytics.nn.modules.conv.Conv	[256, 512, 3, 2]
	-1	1	1380352	ultralytics.nn.modules.block.C3k2	[512, 512, 1, True]
	-1	1	990976	ultralytics.nn.modules.block.C2PSA	[512, 512, 1]
	-1	1	665606	ultralytics.nn.modules.head.Classify	[512, 6]

```
Summary: 86 layers, 5,450,694 parameters, 5,450,694 gradients, 12.3 GFLOPs
```

```
34/236 items from pretrained weights
```

```
Automatic Mixed Precision (AMP) checks...
```

```
passed ✅
```

```
Image access ✅ (ping: 0.0±0.0 ms, read: 314.4±148.8 MB/s, size: 8.6 KB)
```

```
Using /content/garbage_dataset/train... 11120 images, 0 corrupt: 100% — 11120/11120 : 111.1MiB
```

```
Image access ✅ (ping: 0.0±0.0 ms, read: 216.8±74.4 MB/s, size: 6.7 KB)
```

```
/content/garbage_dataset/val... 2781 images, 0 corrupt: 100% — 2781/2781 111.1MiB
```

```
'optimizer=auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and determining best 'optimizer=adamW(lr=0.001, momentum=0.9) with parameter groups 39 weight(decay=0.0), 40 weight(decay=0.0001)
```

```
24 train, 224 val
```

```
16 worker workers
```

```
Transferring to /content/runs/classify/train-2
```

```
Training for 20 epochs...
```

GPU_mem	loss	Instances	Size
0.756G	0.884	16	224: 100% — 348/348 4.5it/s 1:18
classes	top1_acc	top5_acc	100% — 44/44 5.4it/s 8.1s
all	0.881	0.997	

GPU_mem	loss	Instances	Size	
0.934G	0.4509	16	224: 100%	348/348 4.8it/s 1:12
classes	top1_acc	top5_acc: 100%		44/44 6.6it/s 6.7s
all	0.876	0.999		

GPU_mem	loss	Instances	Size	
0.934G	0.4863	16	224: 100%	348/348 4.9it/s 1:11
classes	top1_acc	top5_acc: 100%		44/44 6.7it/s 6.6s
all	0.862	0.998		

GPU_mem	loss	Instances	Size	
0.934G	0.4403	16	224: 100%	348/348 4.9it/s 1:11
classes	top1_acc	top5_acc: 100%		44/44 6.7it/s 6.5s
all	0.883	0.997		

GPU_mem	loss	Instances	Size	
0.934G	0.3495	16	224: 100%	348/348 4.9it/s 1:11
classes	top1_acc	top5_acc: 100%		44/44 6.6it/s 6.6s

✓ Export and Save Trained Weights

Download the best-performing PyTorch model weights (`best.pt`) to the local machine.

```
from google.colab import files

# download the model file to your local computer
files.download('/content/runs/classify/train/weights/best.pt')
```

✓ Model Evaluation & Validation

Load the saved weights to validate performance metrics (Top-1 accuracy) on the validation set.

```
from ultralytics import YOLO

# Load the best trained model
model = YOLO("/content/runs/classify/train/weights/best.pt")
```

```
# Evaluate the model on the validation dataset
metrics = model.val()
# Display the Top-1 and Top-5 accuracy
metrics.top1
metrics.top5
```

```
Ultralytics 8.4.118 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO26s-cls summary (fused): 47 layers, 5,441,814 parameters, 0 gradients, 12.1 GFLOPs
train: /content/garbage_dataset/train... found 11120 images in 6 classes ✓
val: /content/garbage_dataset/val... found 2781 images in 6 classes ✓
test: None...
```

```
val: Fast image access ✓ (ping: 0.0±0.0 ms, read: 255.4±85.4 MB/s, size: 6.7 KB)
val: Scanning /content/garbage_dataset/val... 2781 images, 0 corrupt: 100% 2781/
      classes top1_acc top5_acc: 100% 174/174 18.8it/s 9.3s
           all    0.947    0.999
Speed: 0.1ms preprocess, 1.4ms inference, 0.0ms loss, 0.0ms postprocess per image
Results saved to /content/runs/classify/val
0.9467817544937134
```

✓ Run Inference on Test Images

Pass sample images through the trained model to perform classification predictions.

```
# Install the Ultralytics library for YOLO
!pip install -q ultralytics

from ultralytics import YOLO

# Load the trained model
model = YOLO("/content/best.pt")
# Make a prediction on the test image
results = model("/content/trash1.jpg")
```

```
45.5/45.5 kB 2.6 MB/s eta 0:00:00
1.4/1.4 MB 30.5 MB/s eta 0:00:00
53.2/53.2 kB 3.9 MB/s eta 0:00:00
Creating new Ultralytics Settings v0.0.7 file ✓
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For
image 1/1 /content/trash1.jpg: 224x224 trash 1.00, glass 0.00, cardboard 0.00, paper 0.00, plastic 0.00
Speed: 34.4ms preprocess, 253.0ms inference, 0.1ms postprocess per image at shape (1, 3, 224, 224)
```

```
top1_id = results[0].probs.top1 # Get predicted class index
top1_label = results[0].names[top1_id] # Look up the string label

print(f"Predicted class: {top1_label.upper()}")
```

Predicted class: TRASH

```
results[0].names
```

```
{0: 'cardboard', 1: 'glass', 2: 'metal', 3: 'paper', 4: 'plastic', 5: 'trash'}
```

```
results[0].show()
```

trash 1.00
glass 0.00
cardboard 0.00
paper 0.00
plastic 0.00



```
results = model("/content/trash.jpg")  
for result in results:  
    result.show()
```

image 1/1 /content/trash.jpg: 224x224 metal 0.52, glass 0.38, trash 0.07, plastic 0.03, cardboard 0.01
Speed: 3.7ms preprocess, 5.3ms inference, 0.0ms postprocess per image at shape (1, 3, 224, 224)

metal 0.52
glass 0.38
trash 0.07
plastic 0.03
cardboard 0.01



✓ Helper Function for Inference

Define a reusable `predict()` function that accepts an image path, runs inference using the trained model, prints the top predicted class, and displays the annotated result image.

```
def predict(img_path):  
    results = model(img_path)  
    for result in results:  
        pred_id = result.probs.top1  
        pred_label = result.names[pred_id]  
        print(f"Predicted class: {pred_label.upper()}")  
        result.show()
```

```
predict("/content/card.jpg")
```

image 1/1 /content/card.jpg: 224x224 cardboard 1.00, paper 0.00, trash 0.00, plastic 0.00, glass 0.00
Speed: 7.7ms preprocess, 56.0ms inference, 0.1ms postprocess per image at shape (1, 3, 224, 224)
Predicted class: CARDBOARD



✓ Export Model to ONNX Format

Convert the trained PyTorch model (`best.pt`) to ONNX format (`best.onnx`) for optimized deployment and reduced file size.

```
from ultralytics import YOLO  
  
model = YOLO("/content/best.pt")  
model.export(format="onnx")
```

Ultralytics 8.4.120 🚀 Python-3.12.13 torch-2.11.0+cpu CPU (Intel Xeon CPU @ 2.20GHz)
YOLO26s-cls summary (fused): 47 layers, 5,441,814 parameters, 0 gradients, 1.5 GFLOPs

PyTorch: starting from '/content/best.pt' with input shape (1, 3, 224, 224) BCHW and output s

ONNX: starting export with onnx 1.22.0 opset 20...

ONNX: slimming with onnxslim 0.1.95...

ONNX: export success ✅ 0.8s, saved as '/content/best.onnx' (20.8 MB)

Export complete (0.9s)

Results saved to **/content/best.onnx**

Predict: yolo predict task=classify model=/content/best.onnx imgsz=224

Validate: yolo val task=classify model=/content/best.onnx imgsz=224 data=/content/garba

Visualize: <https://netron.app>

'/content/best.onnx'

Start coding or [generate](#) with AI.